

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

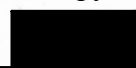
«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Реверс-инжиниринг проверки пароля в исполняемом файле

Выполнили:

Жук Анастасия Валерьевна,
студентка группы N3348



(подпись)

Проверил:

Ищенко Алексей Петрович

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	4
ЗАКЛЮЧЕНИЕ.....	13

ВВЕДЕНИЕ

Цель работы – исследование механизма аутентификации исполняемого файла путем обратной инженерии (реверс-инжиниринга) с целью восстановления корректного пароля на основе предоставленной «соли» и алгоритма проверки, реализованного в программе.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- проанализировать логику работы программы, в том числе механизм запроса пароля и использования «соли»;
- изучить алгоритм проверки пароля, реализованный в исполняемом файле, без применения бинарного патчинга;
- определить набор допустимых символов пароля (строчные английские буквы и цифры);
- разработать метод вычисления или подбора пароля с учетом ограничения на количество попыток ввода;
- получить корректный пароль для входа в программу и подтвердить его работоспособность.

ХОД РАБОТЫ

В рамках лабораторной работы представлено исполняемое приложение, реализующее механизм аутентификации пользователя. При запуске программа отображает графический интерфейс, содержащий открытый параметр – секрет («соль») – и поле для ввода пароля. Задачей является определение корректного пароля на основе предоставленной «соли» и внутренней логики работы программы, без модификации ее исполняемого кода.

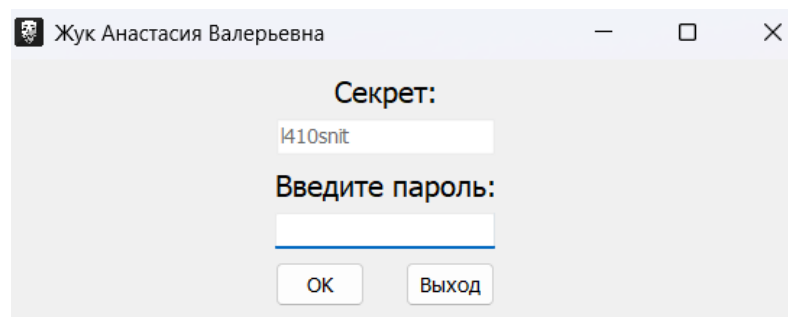


Рисунок 1 – Графический интерфейс приложения

Попробуем ввести какой-нибудь пароль.

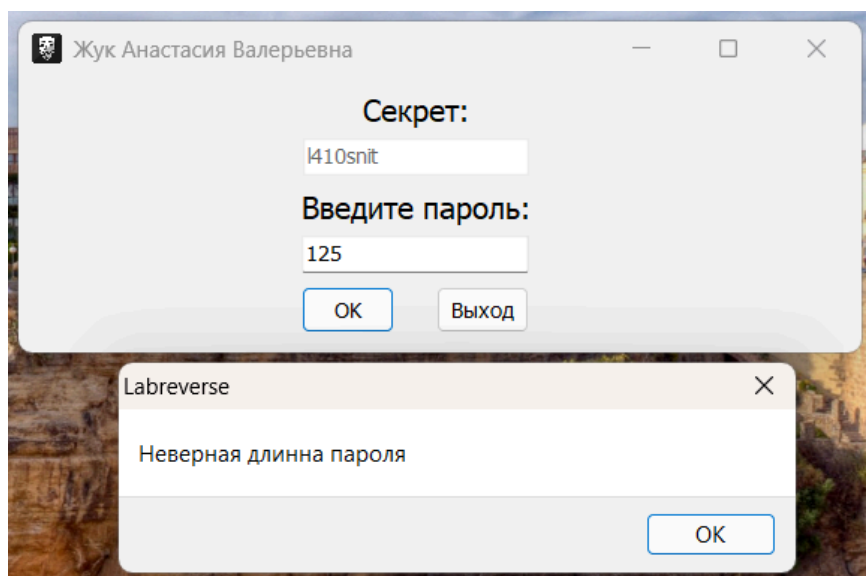


Рисунок 2 – Результат ввода randomного пароля

Итак, мы не угадали с длиной. Это наша первая зацепка. Попробуем перебрать пароли различных длин, начиная с одного символа и на каждом этапе прибавляя один символ. Введя последовательность

12345678, программа уже отреагировала по-другому: теперь у нас просто не верный пароль.

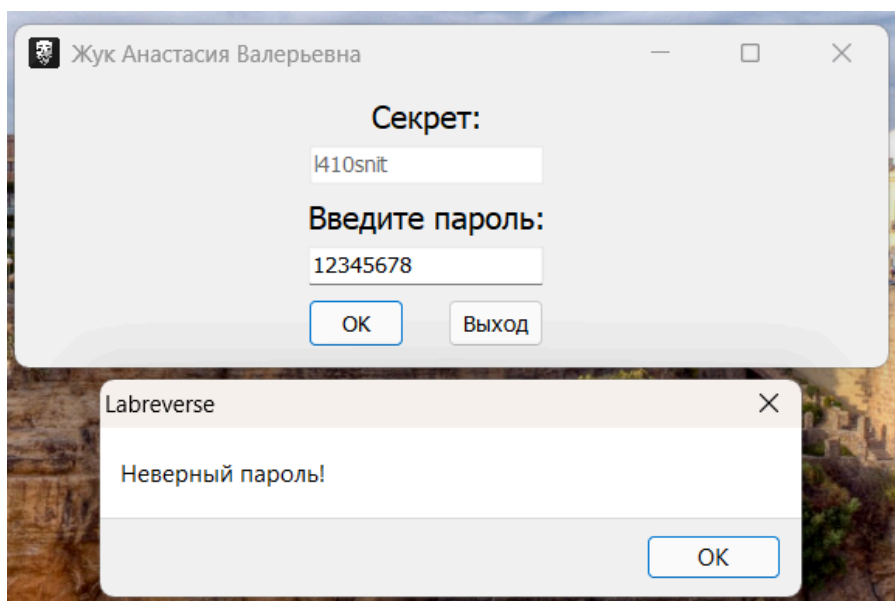


Рисунок 3 – Нашелся пароль схожей длины с правильным

Значит, длина исходного сообщения составляет 8 символов. Перебирать руками не самый лучший вариант: во-первых, это очень долго, а во-вторых, через n -ое число неверно введенных паролей появляется новое окно, в котором написано «Вы заблокированы!» и обратный отсчет 10 секунд. Когда 10 секунд прошли, снова открывается исходное окно и можно вновь попытать удачу. Этот лимит на число попыток ввода пароля остановит нас, если мы захотим написать код, который будет перебирать все возможные пароли длины 8.

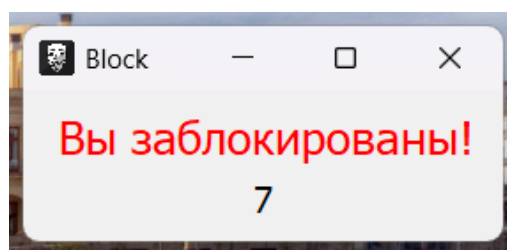


Рисунок 4 – Лимит на число попыток ввода пароля

Поэтому нужен более умный способ поиска этого пароля. Заглянем под капот приложения: посмотрим ассемблеровский код через дизассемблер. Я буду использовать для анализа x32dbg.

Итак, снова пробуем запустить наше приложение и вводим в него пароль нужной длины. Теперь открываем x32dbg и присоединяем открытое приложение (см. рисунок 5).

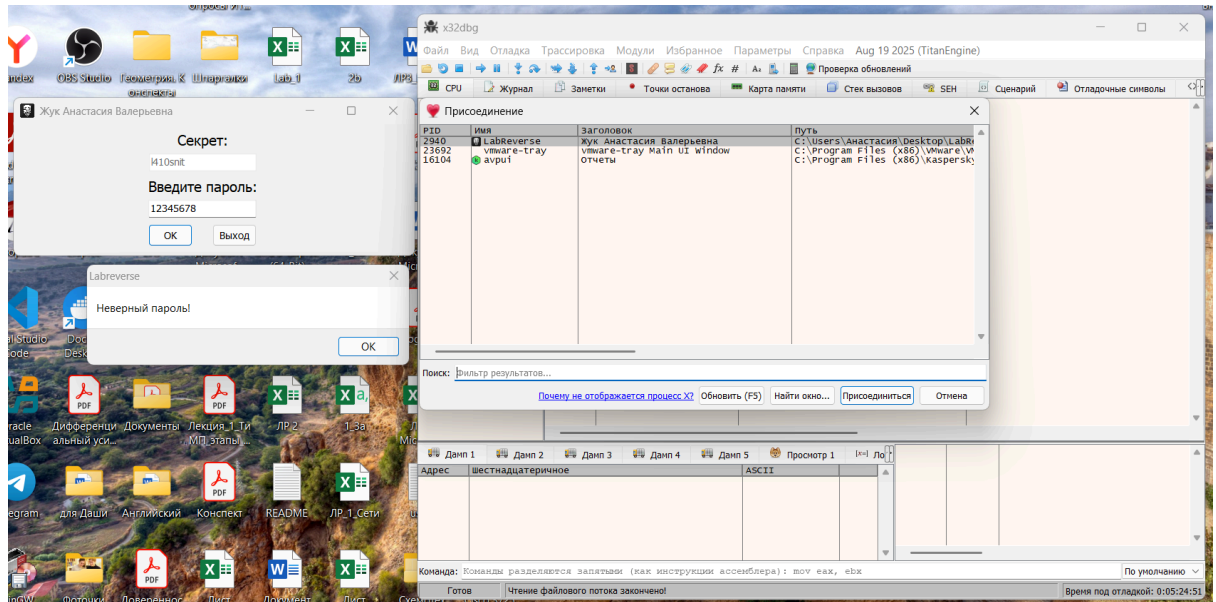


Рисунок 5 – Присоединения запущенного приложения к дизассемблеру

Дальше переходим во вкладку «Отладочные символы» и находим там наш модуль «labreverse.exe». Дважды кликаем на него и получаем ассемблер нашего приложения.

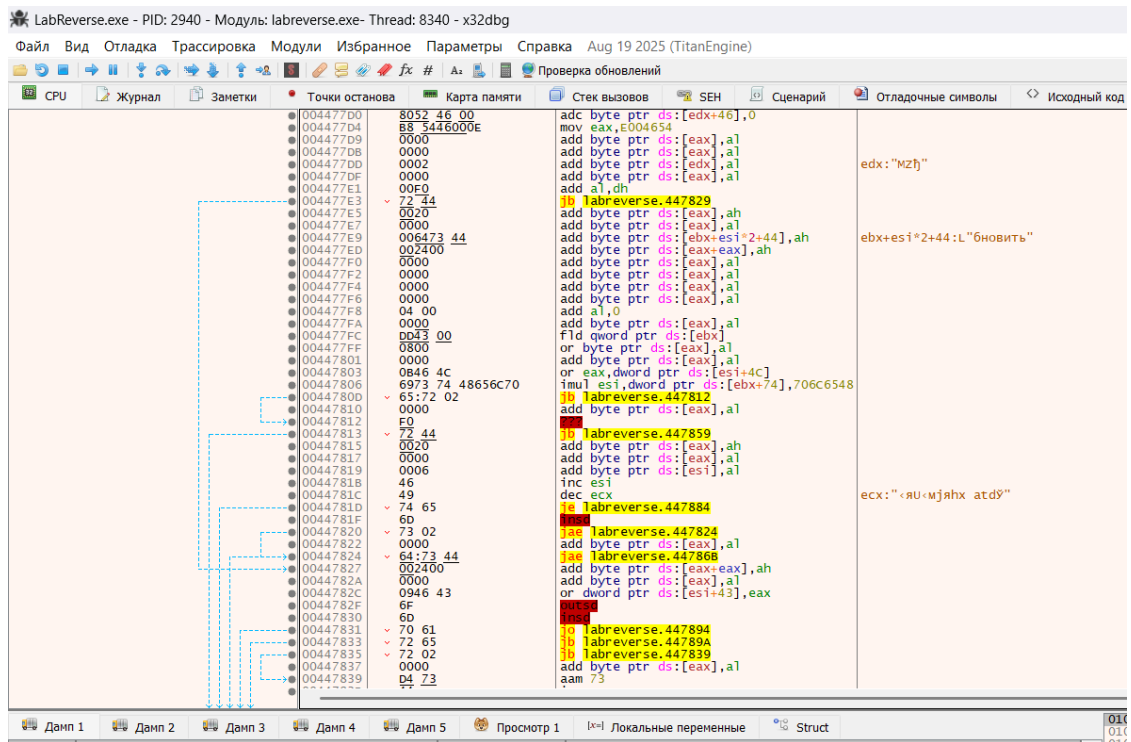


Рисунок 6 – Ассемблер нашего приложения

Теперь правой кнопкой мыши нажмем на ассемблероский код и перейдем туда, куда показано на рисунке 7.

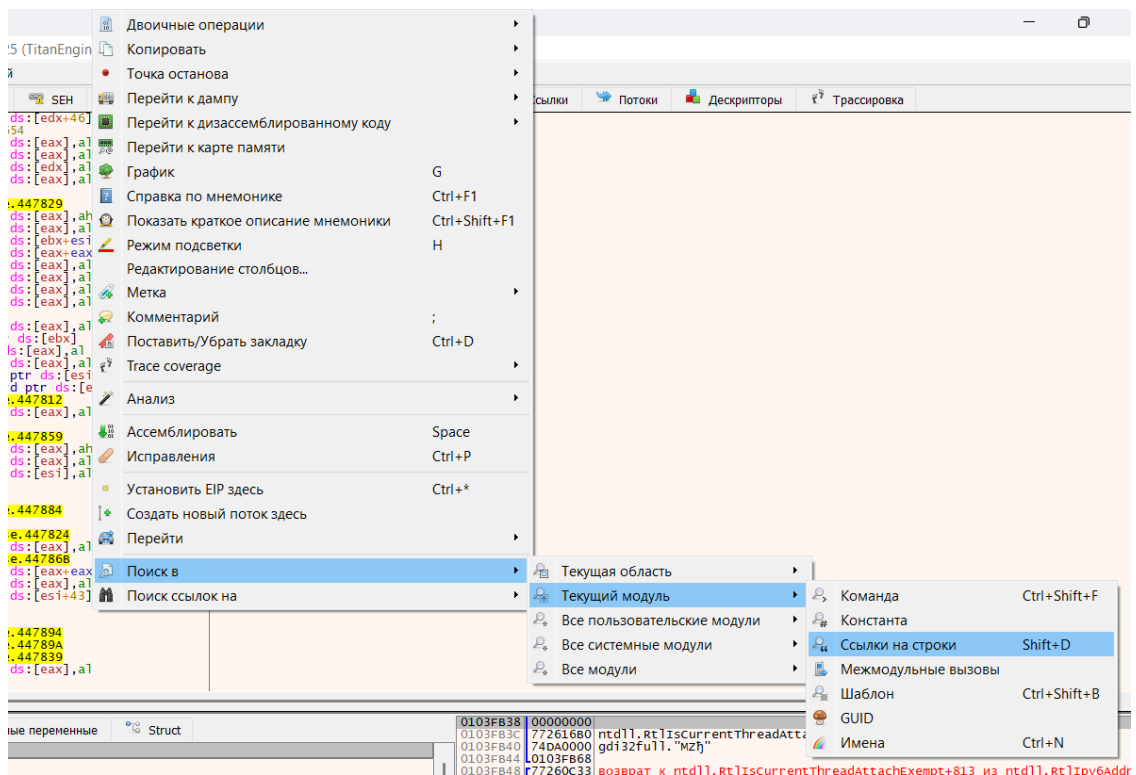


Рисунок 7 – Открываем ссылки на строки нашего ассемблера

И нам открываются строки кода, в которых лежат какие-то данные, в том числе и нужные нам (среди этих строк наш пароль).

Адрес	Дизассемблированный код	Адрес	Строка
0040032D	shl byte ptr ds:[C60000],1	00C60000	"????????????????????????????"
00401031	add eax,labreverse.64F3F0	0064F3F0	"S"jf"
004010A3	adc eax,labreverse.6579A4	006579A4	"йгяя"
00402769	mov edx,dword ptr ds:[668E70]	00668E70	"-f"
004027C3	mov edx,dword ptr ds:[668E74]	00668E74	"r-f"
004027D1	mov edx,dword ptr ds:[668E7C]	00668E7C	"A-f"
004027DE	mov esi,dword ptr ds:[668E78]	00668E78	"&"h4B"
00402A18	push labreverse.402AAD	00402AAD	"&"r -"
00402C50	inc dword ptr ds:[669620]	00669620	"юяяя"
00402C60	sub dword ptr ds:[669620],1	00669620	"юяяя"
00402C81	mov edx,dword ptr ds:[6695BC]	006695BC	"xpf"
00402C8E	mov edx,dword ptr ds:[6695BC]	006695BC	"xpf"
00402C90	mov ecx,dword ptr ds:[6695BC]	006695BC	"xpf"
00402CA5	mov ecx,dword ptr ds:[<&_Form1>]	00668E50	"(-f"
00402CAB	mov edx,dword ptr ds:[65DA80]	0065DA80	"&"p%S"
00402CB6	mov eax,dword ptr ds:[6695BC]	006695BC	"xpf"
00402CBD	mov ecx,dword ptr ds:[<&_Form2>]	00668E54	"p-f"
00402CC3	mov edx,dword ptr ds:[65DD44]	0065DD44	"&"p%S"
00402CC5	mov eax,dword ptr ds:[6695BC]	006695BC	"xpf"
00402CD5	mov ecx,dword ptr ds:[<&_Form3>]	00668E58	"x-f"
00402CD8	mov edx,dword ptr ds:[65E044]	0065E044	"&"p%S"
00402CE6	mov eax,dword ptr ds:[6695BC]	006695BC	"xpf"
00402CFD	mov edx,dword ptr ds:[6695BC]	006695BC	"xpf"
00402D46	mov eax,dword ptr ds:[41CD20]	0041CD20	"хНА"
00402D5E	mov edx,dword ptr ds:[6695BC]	006695BC	"xpf"
00402F88	push labreverse.4042A4	004042A4	"U<мГдЬёрще"
0040317E	push labreverse.65D29D	0065D29D	"8M=Ay?9B"
00403196	push labreverse.66963C	0066963C	"&"8M=Ay?9B"
004033D3	push labreverse.65D2C1	0065D2C1	"Неверная длина пароля"
00403563	push labreverse.65D2DD	0065D2DD	"00000000"
00403612	push labreverse.65D2E6	0065D2E6	"00000000"
00403861	push labreverse.65D2F0	0065D2F0	"00000000"
00403910	push labreverse.65D2F9	0065D2F9	"00000000"
0040403E	push labreverse.66963C	0066963C	"&"8M=Ay?9B"
00404088	mov eax,dword ptr ds:[<&_Form2>]	00668E54	"p-f"
004040B4	mov ecx,dword ptr ds:[<&_Form2>]	00668E54	"p-f"
004040CF	push labreverse.65D303	0065D303	"Неверный пароль!"
00404124	mov eax,dword ptr ds:[4FE530]	004FE530	"ёо"
00404156	push labreverse.65D314	0065D314	"Micro-Software"
004041B8	push labreverse.65D323	0065D323	"soft"
00404211	mov ecx,dword ptr ds:[<&_Form3>]	00668E58	"x-f"
0040421E	mov eax,dword ptr ds:[<_Form1>]	00669628	"&"шьё"
0040422A	mov eax,dword ptr ds:[<_Form1>]	00669628	"&"шьё"
00404350	push labreverse.65D362	0065D362	"L"%i"
004043D6	push labreverse.65D368	0065D368	"L"%c"
0040441C	push labreverse.65D32A	0065D32A	"жук Анастасия Валерьевна"
00404432	mov eax,dword ptr ds:[<_Form1>]	00669628	"&"шьё"
00404453	push labreverse.65D343	0065D343	"1410snit"
0040448A	mov eax,dword ptr ds:[4FE530]	004FE530	"ёо"
004044AA	push labreverse.65D34C	0065D34C	"Micro-Software"
0040450F	push labreverse.65D35C	0065D35C	"soft"
004045A1	mov ecx,dword ptr ds:[<&_Form3>]	00668E58	"x-f"

Рисунок 8 – Строки файла labreverse.exe

Кликнем дважды по строке «Неверная длина пароля» и увидим тот участок в ассемблеровском коде, в котором приложение должно выдавать ЭТОТ ОТВЕТ.

004033C8	74 40	je labreverse.40340A	24: '\$'
004033CA	66:C785 38FFFFFF 240C	mov word ptr ss:[ebp-C8],24	65D2C1: "Неверная длина пароля"
004033D3	68 C1D26500	push labreverse.65D2C1	[ebp-0C]: "МЗђ"
004033D8	8D4D F4	lea ecx,dword ptr ss:[ebp-C]	ecx: "<яУ<мјяћх atdŸ"
004033DB	51	push ecx	
004033DC	E8 73802500	call labreverse.65B454	
004033E1	83C4 08	add esp,8	
004033E4	FF85 44FFFFFF	inc dword ptr ss:[ebp-BC]	[ebp-BC]: "<ш...яш...<Ua<}>ш<f'r"
004033EA	8B00	mov eax,dword ptr ds:[eax]	
004033EC	E8 9B7C1B00	call labreverse.5B808C	
004033F1	FF8D 44FFFFFF	dec dword ptr ss:[ebp-BC]	[ebp-BC]: "<ш...яш...<Ua<}>ш<f'r"
004033F7	6A 02	push 2	
004033F9	8D55 F4	lea edx,dword ptr ss:[ebp-C]	[ebp-0C]: "МЗђ"
004033FC	52	push edx	edx: "МЗђ"
004033FD	E8 FE802500	call labreverse.65B500	
00403402	83C4 08	add esp,8	
00403405	E9 860E0000	jmp labreverse.404290	
0040340A	33C9	xor ecx,ecx	
0040340C	898D 1CFFFFFF	mov word ptr ss:[ebp-E4],ecx	ecx: "<яУ<мјяћх atdŸ"
00403412	8B85 1CFFFFFF	mov eax,dword ptr ss:[ebp-E4]	
00403418	0105 24D26500	add dword ptr ds:[65D224],eax	
0040341E	833D 24D26500 07	cmp dword ptr ds:[65D224],7	
00403425	7E 07	jle labreverse.40342E	
00403427	8305 24D26500 F8	add dword ptr ds:[65D224],FFFFFFF8	
0040342E	FF85 1CFFFFFF	inc dword ptr ss:[ebp-E4]	
00403434	83BD 1CFFFFFF 08	cmp dword ptr ss:[ebp-E4],8	
0040343B	7C D5	jl labreverse.403412	

Рисунок 9 – Участок кода приложения, где оно выводит сообщение «Неверная длина пароля»

Теперь нужно понять, а как вообще сюда попадает приложение, какие действия оно предпринимает при обработке введенной информации, чтобы прийти к выводу, что у пароля неверная длина. Так как код сам по себе очень большой, существует очень удобное представление такого кода в виде графа.

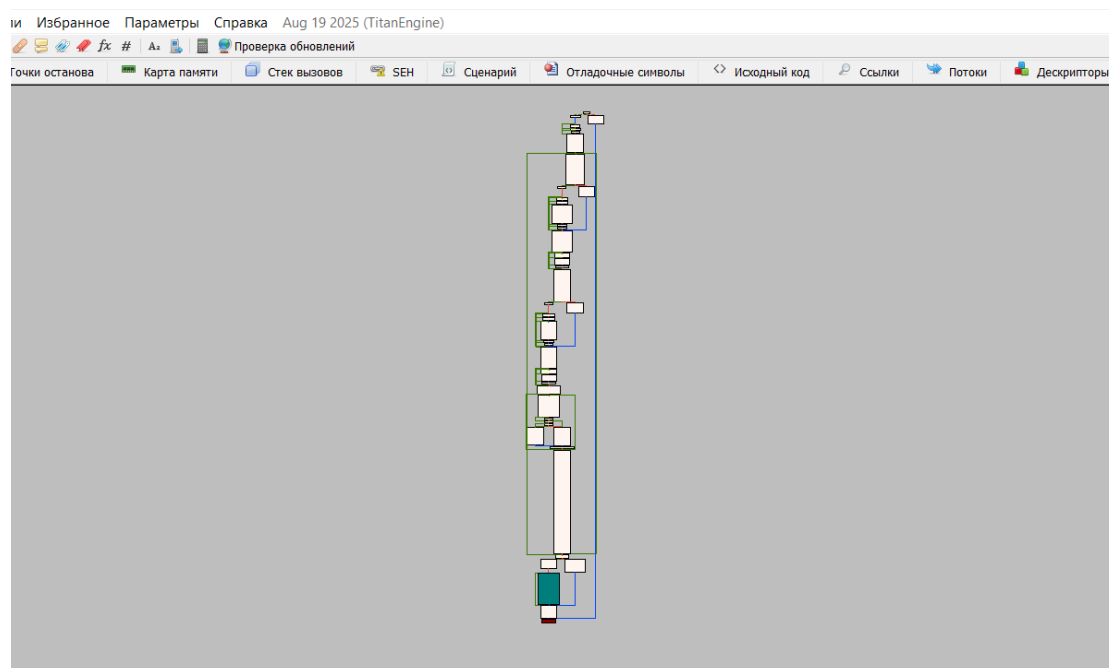


Рисунок 10 – Общий вид графа

Попробуем поискать в нем те участки кода, которые помогут нам понять, как выглядит засекреченный пароль.

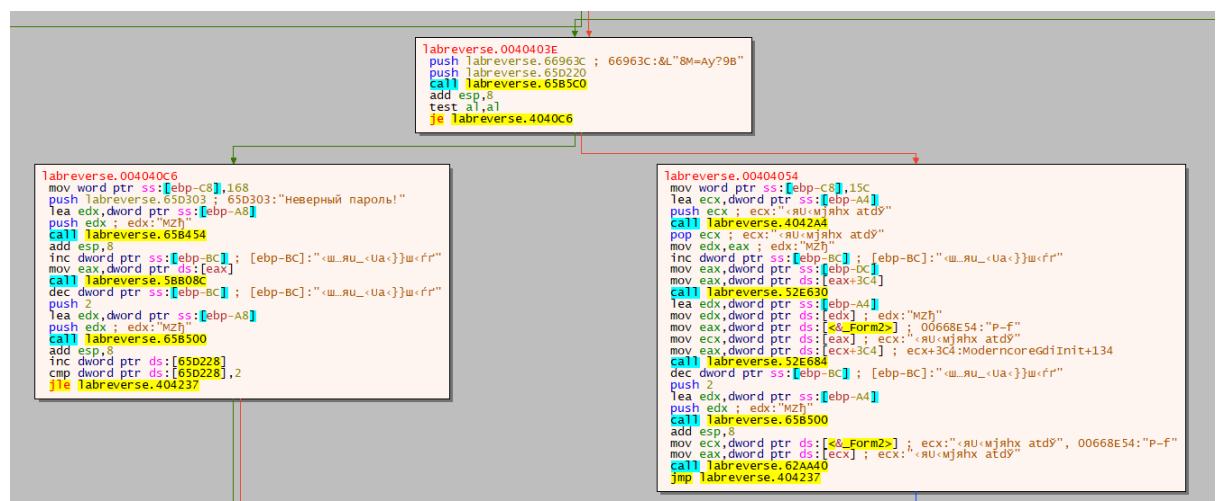
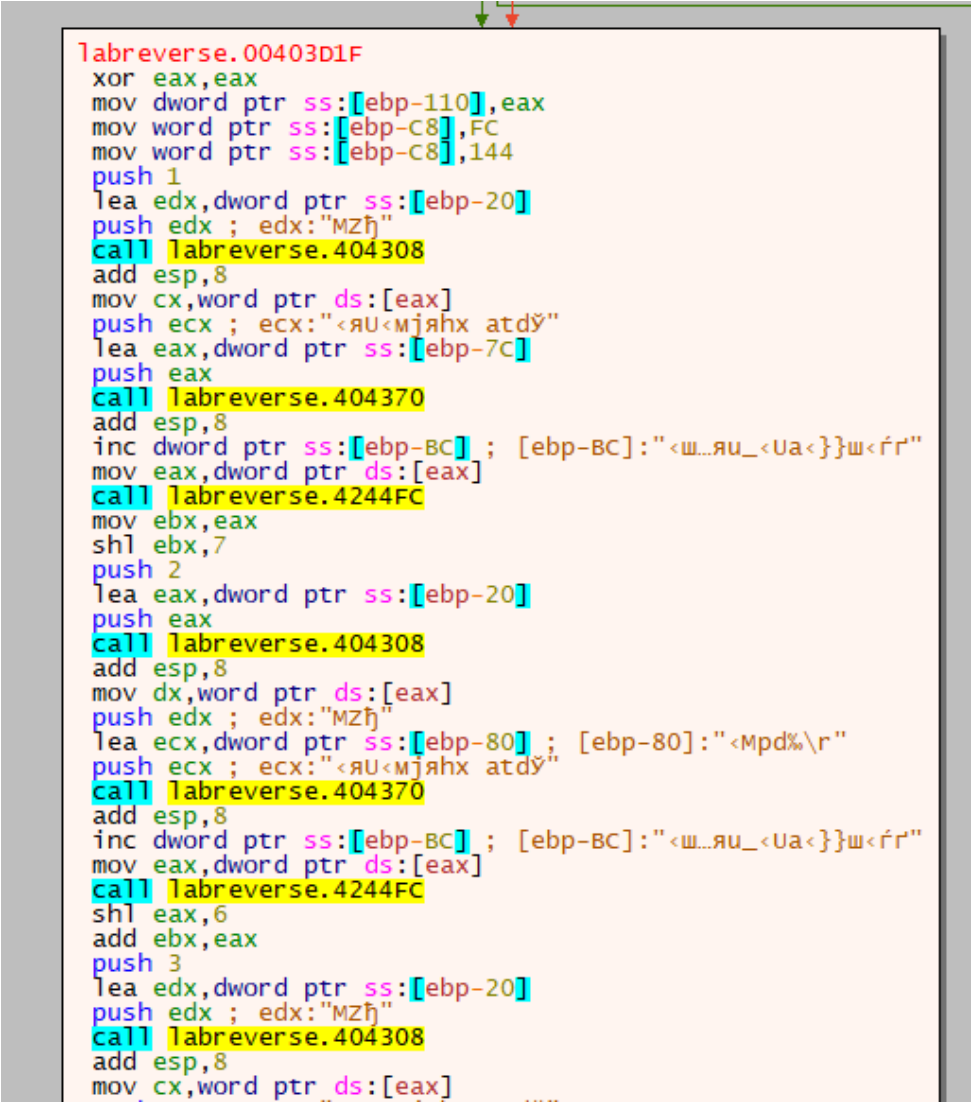


Рисунок 11 – Ветвление графа при соотношении засекреченных введенных данных и правильного пароля

На рисунке 11 видно, как программа сравнивает введенные данные, захэшированные с помощью секрета, со строкой 8M=Ay?9B. Значит, эта строка и есть наш захэшированный пароль. Теперь нужно понять, как получить сам пароль. Снова обратимся к графу и попробуем найти момент, когда введенная нами строка хэшировалась с помощью секрета. Идем по нему выше, находим огромный блок со сдвигами, хог и в самом начале сдвиг на [ebp-F8], то есть на 48 символов. И находим, спускаясь чуть ниже коды на +48 и -48). При XOR это надо для того, чтобы преобразовывать по ASCII таблице из символа в число и обратно, для усложнения задач по подбору пароля.



```

labreverse.00403D1F
xor eax,eax
mov dword ptr ss:[ebp-110],eax
mov word ptr ss:[ebp-C8],FC
mov word ptr ss:[ebp-C8],144
push 1
lea edx,dword ptr ss:[ebp-20]
push edx ; edx:"MZh"
call labreverse.404308
add esp,8
mov cx,word ptr ds:[eax]
push ecx ; ecx:"<яU<мjяhx atdŷ"
lea eax,dword ptr ss:[ebp-7C]
push eax
call labreverse.404370
add esp,8
inc dword ptr ss:[ebp-BC] ; [ebp-BC]:"<ш...яU<Ua<}}ш<ѓг"
mov eax,dword ptr ds:[eax]
call labreverse.4244FC
mov ebx,eax
shl ebx,7
push 2
lea eax,dword ptr ss:[ebp-20]
push eax
call labreverse.404308
add esp,8
mov dx,word ptr ds:[eax]
push edx ; edx:"MZh"
lea ecx,dword ptr ss:[ebp-80] ; [ebp-80]:"<mpd%\r"
push ecx ; ecx:"<яU<мjяhx atdŷ"
call labreverse.404370
add esp,8
inc dword ptr ss:[ebp-BC] ; [ebp-BC]:"<ш...яU<Ua<}}ш<ѓг"
mov eax,dword ptr ds:[eax]
call labreverse.4244FC
shl eax,6
add ebx,eax
push 3
lea edx,dword ptr ss:[ebp-20]
push edx ; edx:"MZh"
call labreverse.404308
add esp,8
mov cx,word ptr ds:[eax]

```

Рисунок 12 – Огромный модуль со сдвигами (часть)

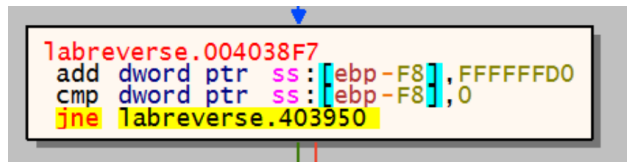


Рисунок 13 – Самое начало с преобразованием в число с помощью сдвига на 48

Принцип формирования понятен. Напише код на языке программирования Python, подставим туда наш секрет и найденную строку. Запустим и получим искомый пароль.

```

timp_lab4.py x
1 usage
2 def restore_password(secret, correct_hash):
3     password_chars = []
4
5     for secret_char, hash_char in zip(secret, correct_hash):
6         xor_key = ord(secret_char) - 48
7         decoded_char = ord(hash_char) ^ xor_key
8         password_chars.append(chr(decoded_char + 48))
9
10    return ''.join(password_chars)
11
12    result = restore_password("l410snit", "8M=Ay?9B")
13    print(result)

timp_lab4 x
:
C:\py.project\first\venv\Scripts\python.exe C:\py.project\first\projects\timp_lab4.py
4ylqj106

```

Рисунок 14 – Поиск искомого пароля

Подставим в наше приложение и убедимся, что то, что мы нашли, действительно является паролем.

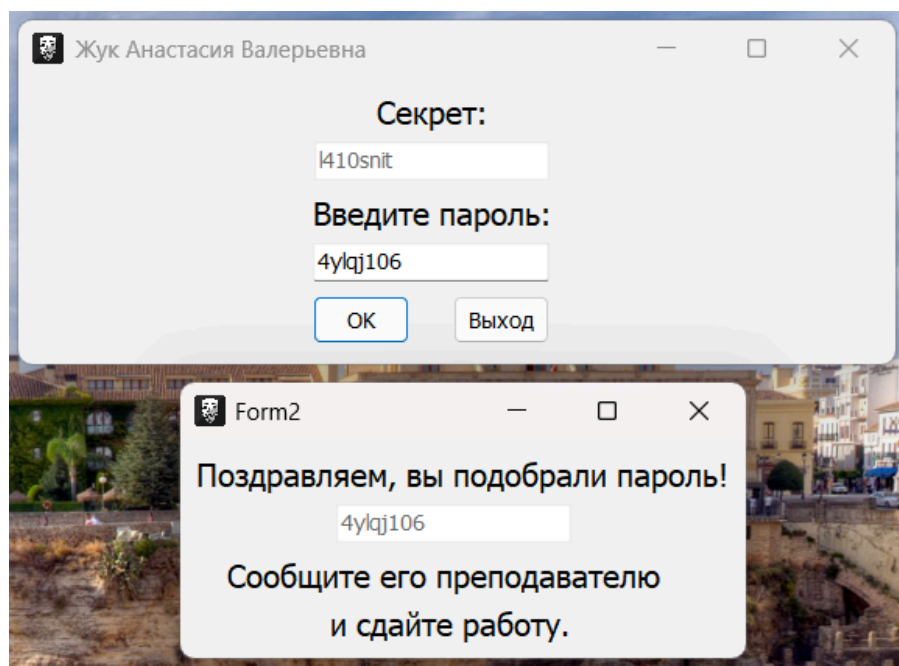


Рисунок 15 – Введен правильный пароль, система взломана
На рисунке 15 видно, что у нас все получилось!

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был успешно проведен анализ механизма проверки пароля в исполняемом файле с применением методов реверс-инжиниринга. Используя отладчик x32dbg, удалось исследовать ассемблерный код программы, определить алгоритм преобразования введенных данных с использованием «соли» и восстановить логику сравнения с эталонным значением. На основе изученного алгоритма был разработан Python-скрипт, позволивший вычислить корректный пароль без нарушения ограничений на количество попыток ввода. Полученный пароль был верифицирован — его ввод обеспечил успешный доступ к программе. Работа продемонстрировала эффективность применения инструментов статического и динамического анализа для исследования закрытых алгоритмов аутентификации и подтвердила возможность восстановления защищенных данных при корректном подходе к обратной инженерии.